

OE-EC804A — Artificial Intelligence

Semester VIII • 3L:0T:0P • 3 Credits

Module 2: Solving Problems by Searching (Uninformed Search)

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Problem-Solving Agents & Problem Formulation	2
1.1	State Space and Search Trees	2
1.2	Evaluation Criteria for Search Algorithms	2
2	Uninformed Search Strategies	2
2.1	1. Breadth-First Search (BFS)	3
2.2	2. Depth-First Search (DFS)	3
2.3	3. Depth-Limited Search (DLS)	3
2.4	4. Iterative Deepening DFS (IDDFS)	3
2.5	5. Bidirectional Search	4
3	Comparison of Search Strategies	4

1. Problem-Solving Agents & Problem Formulation

A **Problem-Solving Agent** is a goal-based agent that decides what to do by finding sequences of actions that lead to desirable states.

Definition of a Problem

A **Problem** is formally defined by 5 components:

1. **Initial State:** $s_0 \in S$, the state where the agent begins.
2. **Actions:** $Actions(s)$, the set of valid actions available in state s .
3. **Transition Model:** $Result(s, a)$, returns the outcome state resulting from taking action a in state s .
4. **Goal Test:** Function determining whether a given state is a goal state.
5. **Path Cost:** $c(s, a, s')$, numeric cost assigned to taking step a from s to s' .

1.1 State Space and Search Trees

- **State Space:** The set of all reachable states from the initial state via any sequence of actions.
- **Search Space / Tree:** A tree generated by the search algorithm where the root is the initial state and child nodes correspond to actions.
- **Branching Factor (b):** Maximum number of successors (children) of any node.
- **Depth (d):** Depth of the shallowest goal node.
- **Maximum Depth (m):** Maximum length of any path in the state space.

1.2 Evaluation Criteria for Search Algorithms

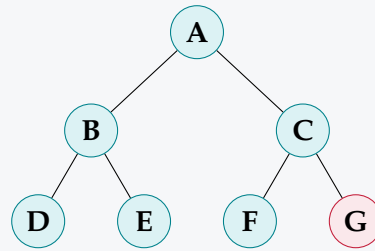
Search algorithms are evaluated based on four fundamental metrics:

1. **Completeness:** Is the algorithm guaranteed to find a solution if one exists?
2. **Time Complexity:** How long (how many nodes generated) does it take to find a solution?
3. **Space Complexity:** How much memory is needed to perform the search?
4. **Optimality:** Does the strategy find the lowest path cost solution?

2. Uninformed Search Strategies

Uninformed search (blind search) strategies have no additional information about states beyond that provided in the problem definition.

Sample Search Tree Topology



2.1 1. Breadth-First Search (BFS)

- **Mechanism:** Expands the root node first, then expands all successors of the root, then their successors. Implemented using a **FIFO (Queue)** frontier.
- **Completeness:** Complete (if branching factor b is finite).
- **Time Complexity:** $O(b^d)$
- **Space Complexity:** $O(b^d)$ (all nodes kept in memory).
- **Optimality:** Optimal if step costs are uniform (equal to 1).

2.2 2. Depth-First Search (DFS)

- **Mechanism:** Expands the deepest node in the current frontier. Implemented using a **LIFO (Stack)** frontier.
- **Completeness:** Incomplete in infinite state spaces or graphs with loops.
- **Time Complexity:** $O(b^m)$
- **Space Complexity:** $O(b \cdot m)$ (linear memory overhead!).
- **Optimality:** Non-optimal (may find a deeper solution before a shallow one).

2.3 3. Depth-Limited Search (DLS)

- **Mechanism:** DFS with a pre-determined depth limit l . Nodes at depth l are treated as if they have no successors.
- **Completeness:** Incomplete if $l < d$.
- **Time Complexity:** $O(b^l)$
- **Space Complexity:** $O(b \cdot l)$
- **Optimality:** Non-optimal if $l > d$.

2.4 4. Iterative Deepening DFS (IDDFS)

- **Mechanism:** Combines the benefits of BFS (completeness/optimality) and DFS (linear memory). It calls DLS repeatedly with increasing depth limits $l = 0, 1, 2, \dots$
- **Completeness:** Complete (if b is finite).
- **Time Complexity:** $O(b^d)$
- **Space Complexity:** $O(b \cdot d)$ (Linear space!).
- **Optimality:** Optimal if step costs are equal.

2.5 5. Bidirectional Search

- **Mechanism:** Runs two simultaneous searches: one forward from initial state and one backward from goal state. Stops when frontiers intersect.
- **Completeness:** Complete if b is finite and both searches are BFS.
- **Time Complexity:** $O(b^{d/2})$
- **Space Complexity:** $O(b^{d/2})$
- **Optimality:** Optimal if step costs are uniform.

3. Comparison of Search Strategies

Criterion	BFS	DFS	IDDFS	Bidirectional
Completeness	Yes (if $b < \infty$)	No	Yes (if $b < \infty$)	Yes (if $b < \infty$)
Time Complexity	$O(b^d)$	$O(b^m)$	$O(b^d)$	$O(b^{d/2})$
Space Complexity	$O(b^d)$	$O(b \cdot m)$	$O(b \cdot d)$	$O(b^{d/2})$
Optimality	Yes (cost = 1)	No	Yes (cost = 1)	Yes (cost = 1)
Frontier Data Struct	FIFO Queue	LIFO Stack	LIFO Stack (Iterative)	Two FIFO Queues

Quick Revision — Module 2

Solving Problems by Searching (Uninformed Search) | OE-EC804A

Key Concepts and Metrics at a Glance

BFS Complexity	Time: $O(b^d)$, Space: $O(b^d)$ (FIFO Queue, Complete, Optimal for step cost = 1)
DFS Complexity	Time: $O(b^m)$, Space: $O(b \cdot m)$ (LIFO Stack, Incomplete, Non-optimal)
IDDFS Complexity	Time: $O(b^d)$, Space: $O(b \cdot d)$ (Linear memory, Complete, Optimal)
Bidirectional Search	Time: $O(b^{d/2})$, Space: $O(b^{d/2})$ (Searches forward & backward)
Problem Components	Initial state s_0 , Actions $Actions(s)$, Transition model $Result(s, a)$, Goal test, Path cost.

One-Line Definitions

- **State Space:** Set of all valid states reachable from initial state.
- **Frontier:** Set of leaf nodes generated but not yet expanded.
- **Branching Factor (b):** Maximum number of successors for any node.
- **IDDFS:** Strategy running DLS with increasing limits $l = 0, 1, 2, \dots$ combining BFS optimality with DFS space.

Frequently Asked / Viva Questions

Q1. What are the 5 components of a formal problem definition?

Initial state, Actions, Transition model, Goal test, and Path cost.

Q2. Compare BFS and DFS in terms of space complexity.

BFS requires $O(b^d)$ space (exponential), whereas DFS requires only $O(b \cdot m)$ space (linear).

Q3. Why is IDDFS preferred over BFS for large state spaces?

IDDFS is complete and optimal like BFS, but uses linear space $O(b \cdot d)$ instead of exponential space $O(b^d)$.

Q4. Why does node regeneration in IDDFS not incur high overhead?

Most nodes in a search tree are at the bottom level (d). Nodes at level d are generated 1 time, level $d - 1$ twice, etc., so total nodes generated $\approx \frac{b}{b-1} b^d$, which is asymptotically $O(b^d)$.

Q5. What is the frontier in search algorithms?

The data structure (queue/stack) containing generated nodes that have not yet been expanded.

Frequently Asked / Viva Questions (contd.)**Q6. When is BFS optimal?**

BFS is optimal when all step costs are equal (uniform path cost).

Q7. Explain the time and space complexity of Bidirectional Search.

Time and space complexities are both $O(b^{d/2})$ because two searches meet in the middle.

Q8. What is Depth-Limited Search and why is it used?

DFS restricted to depth limit l . It prevents DFS from getting stuck in infinite paths.

Q9. What is a Graph Search vs Tree Search?

Graph search maintains an 'explored set' to avoid repeating states in looped graphs. Tree search does not.

Q10. Define Completeness and Optimality.

Completeness: Guaranteed to find a solution if one exists. Optimality: Guaranteed to find the lowest-cost solution.